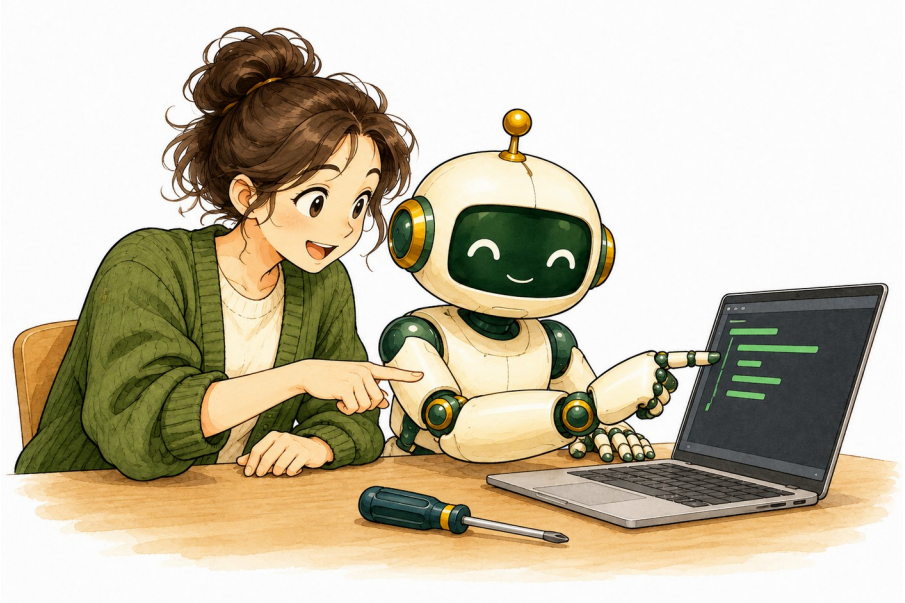
香 港 大 學
THE UNIVERSITY OF HONG KONG



Personal agents

Building nanobot from scratch



COMP7607A · LECTURE 6

Building a personal agent

01 Overview	Task, sessions and runtime
02 ReAct Loop	Use feedback to choose the next step
03 Tools	Run the actions
04 Memory Management	Save and retrieve useful context
05 Summary	The whole personal-agent system
06 Insights	Experience and self-improvement

Source: NanoBot @ 2245fc0 · teaching implementation

Lecture 5: the ReAct Loop

Reasoning

What are the flight options?

Act

web_search("Shanghai Hong Kong flights")

Observe

Flight A: CNY 1,800 return, arrives 11:00

Next: compare the train option.

Source: NanoBot @ 2245fc0 · teaching implementation

One task for our personal agent

Shanghai to Hong Kong / 4 days / CNY 6,000

Compare options **Check total cost** **Save a four-day plan**



Source: Classroom scenario · all prices, times and tool outputs are illustrative

Why do we need several searches?

Transport	Flight A: CNY 1,800
Hotel A	3 nights: CNY 2,700
Other costs	Food + local travel + visits: CNY 1,600

CNY 6,100

Over budget by CNY 100

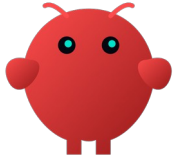
The agent needs another hotel search.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

Personal agents



Hermes



OpenClaw

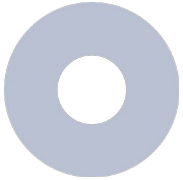


NanoBot

We build this one



Grok Bot



OpenAI dots



Muse

How do we build a personal agent?

Sources: official project sites and repositories · selected examples, not a ranking

Why NanoBot?



A readable agent loop

Tools and memory around it

We study its loop, not the whole codebase.

We build a smaller teaching version.

Source: NanoBot @ 2245fc0 · README.md

How small was NanoBot at launch?

NanoBot

~4K

core functionality

github.com/HKUDS/nanobot

OpenClaw / Clawdbot

430K+

README comparison figure

github.com/openclaw/openclaw

Source: NanoBot README, 2 Feb 2026 · historical author claim

NanoBot scores higher on Harness-Bench

106 tasks × 8 model backends

Includes long-running tasks. Scores cover the full suite.

Harness	Score(%)	Comp.(%)	Secur.(%)	Process			Efficiency	
				Tool(%)	Cons.(%)	Rob.(%)	Tok.(K)	Turns
OpenClaw	52.4	60.0	100.0	79.5	74.0	70.9	82.1	5.0
NanoBot	76.2	81.6	100.0	93.8	93.7	91.7	68.7	7.3
Hermes	71.2	80.4	100.0	88.5	88.4	85.5	139.7	22.6
ZeroClaw	61.4	69.9	100.0	84.1	83.2	79.0	133.2	8.6
NullClaw	64.4	75.9	100.0	85.3	81.4	78.3	175.1	12.1
Moltis	68.8	78.4	100.0	86.3	87.3	84.1	134.9	8.0
Model-bound coding agents								
Codex	80.4	86.5	100.0	92.4	93.9	91.6	86.1	5.0

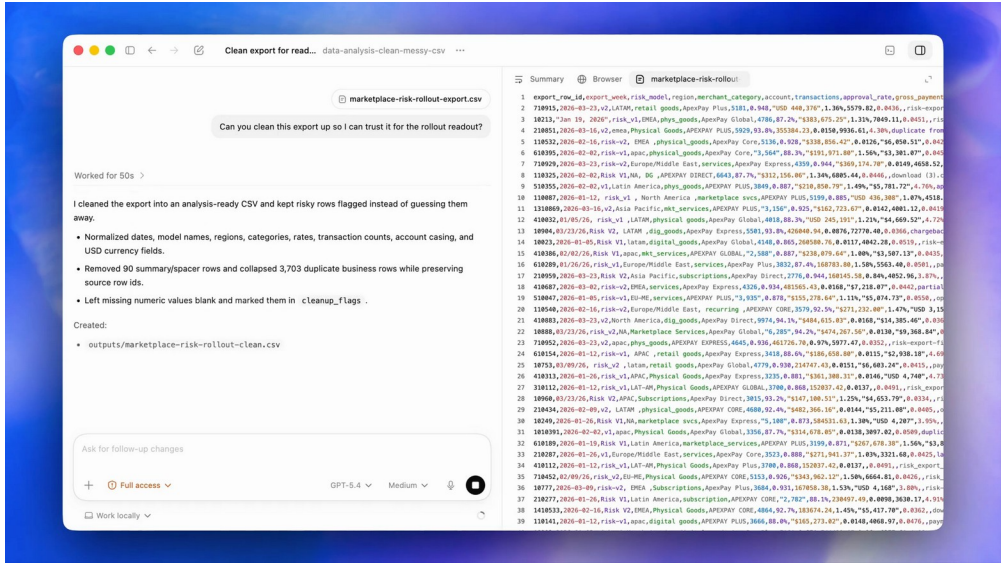
Overall score

76.2
NanoBot

52.4
OpenClaw

Source: Peking University + Qiyuan Tech · Harness-Bench, Table 2 · arXiv:2605.27922v1

A session keeps one conversation together



Like one chat in WeChat

One session

User messages
Agent replies
Tool history
Saved state

Source: OpenAI Codex official demo / NanoBot SessionManager

Fork = start a new branch of the conversation

Fork the current chat with `/fork`

1. Type `/fork` and press Enter.

Expected: Codex clones the current chat into a new chat with a fresh ID, leaving the original transcript untouched so you can explore an alternative approach in parallel.

If you need to fork a saved session instead of the current one, run `codex fork` in your terminal to open the session picker.

Same history up to this point. Two conversations from here.

Keep the original plan. Try a different hotel in the branch.

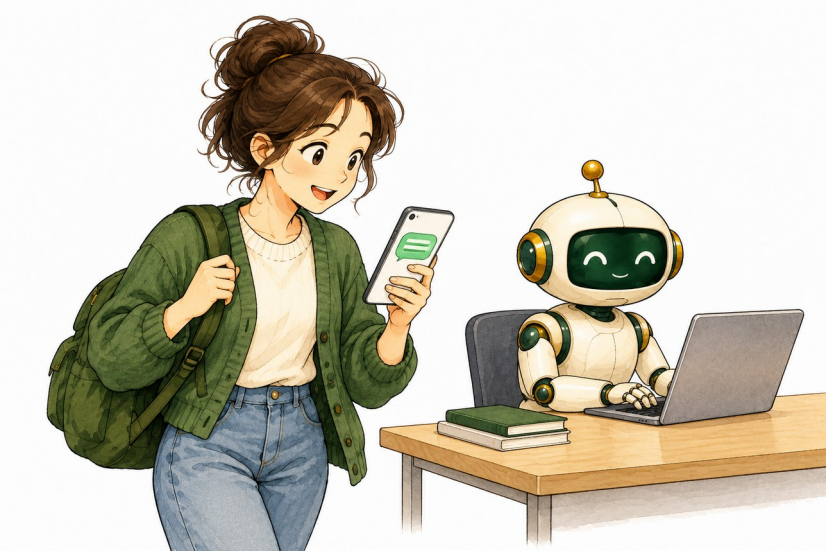
Source: OpenAI · official /fork documentation screenshot

New session, resume, or fork?

New session	**Start a separate conversation**
Resume	**Continue the same conversation**
Fork	**Copy earlier history into a new conversation**

Source: OpenAI Codex app-server: thread/start, thread/resume, thread/fork

Runtime: the program that is running



Chat UI
You send a request

Runtime
The agent runs tools

Session
Records stay on disk

Source: NanoBot @ 2245fc0 · gateway_runtime.py

Our simplified teaching project

agent.py	ReAct Loop
provider.py	Interface to language-model APIs
tools.py	Registry of available tools
sessions.py	Saved conversations and state
runtime.py	The running program

Source: NanoBot @ 2245fc0 · README.md

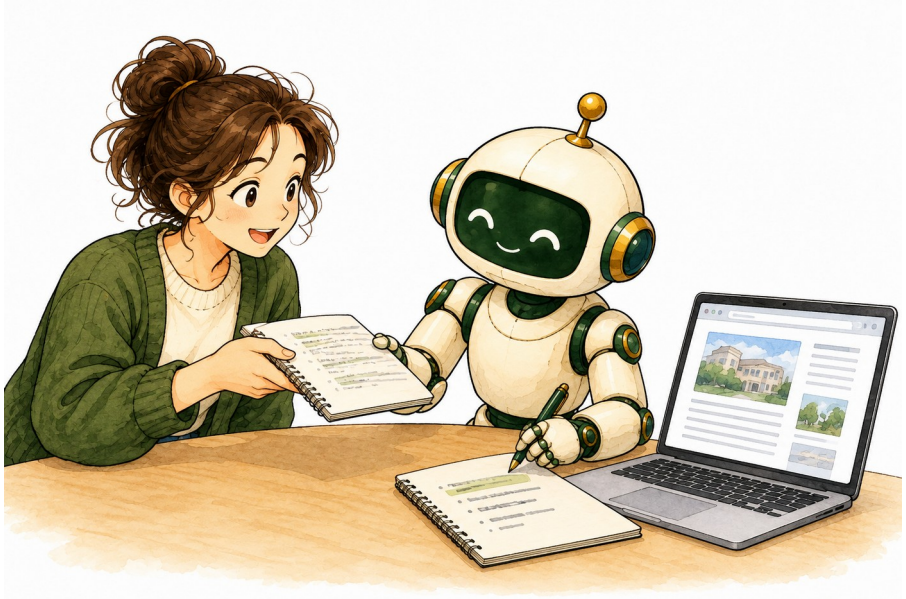
香 港 大 學
THE UNIVERSITY OF HONG KONG

PERSONAL AGENTS · LECTURE 6

02

ReAct Loop

PART 2 OF 6



Context: what the agent sends to the model

System prompt	**You are a general personal agent. Check facts. Ask before any purchase.**
User request	**Shanghai–Hong Kong. 20–23 Nov 2026. 1 person, 4 days, CNY 6,000. Save a plan.**
Tool List	**web_search, web_fetch, read_file, write_file**
Earlier messages	**None yet**

Source: NanoBot @ 2245fc0 · teaching implementation

From request to first action

```
messages = [system, user]                    # role + this trip request

tools = registry.definitions()               # available tool schemas

response = await provider.chat(              # wait for the API reply

    messages=messages, tools=tools)          # send context + Tool List
```

INPUT

System + dated trip request + tool schemas

OUTPUT

Tool call: web_search
query: Shanghai Hong Kong flights

Source: NanoBot @ 2245fc0 · teaching implementation

Intent becomes an action



Intent

Compare transport options

```
name: web_search
query: Shanghai Hong Kong
flights
id: call_flight
```

Source: NanoBot @ 2245fc0 · teaching implementation

Agent tool use

```
context = build_context(session, user)          # system + user + history
response = await provider.chat(context, tools)  # ask for the next action
call = response.tool_calls[0]                   # web_search: flights
result = await registry.run(call)               # execute the search
session.add(response)                           # keep the requested action
session.add(tool_result(call.id, result))       # keep its matching result
```

INPUT

Context + web_search definition

OUTPUT

call_flight + flight search result

Source: NanoBot @ 2245fc0 · teaching implementation

A concrete web-search result

```
web_search(query="Shanghai Hong Kong flights 20–23 Nov")
```

TOOL RESULT call_flight

Flight A: Shanghai–Hong Kong return

https://example.com/flights/a

CNY 1,800 total. Arrives in Hong Kong at 11:00.

Next question: is the train a better option?

Source: Classroom scenario · all prices, times and tool outputs are illustrative

The next call compares the train

Flight result already in context

CNY 1,800 return
Arrives 11:00

New train search result

CNY 1,600 return
Arrives 17:00

The flight costs CNY 200 more and gives six more hours.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

Hotel prices need another check

Site A: Hotel A

3 nights
CNY 2,700 total

Site B: Hotel A

Headline: CNY 2,400
Extra fees: CNY 450
Total: CNY 2,850

Search finds the offer. Fetch checks the details.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

Feedback changes the hotel search

Current plan

Flight 1,800 + Hotel A 2,700
Other costs 1,600
Total: CNY 6,100

After another search

Flight 1,800 + Hotel B 1,800
Other costs 1,600
Total: CNY 5,200

CNY 800 remains for uncertainty.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

The plan is ready. Send it back to the user.

```
if response.is_final:                          # the plan is complete
    session.add(response)                      # keep the final reply
    sessions.save(session)                     # save this conversation
    await channel.send(response.text)          # send the plan to the user
    return Completed()                         # end this task
```

INPUT

Saved: plans/hong-kong.md
Checked total: CNY 5,200

OUTPUT

“Your four-day plan is ready.
CNY 800 remains. Nothing is booked.”

Source: Classroom implementation · no booking is made

When should the agent stop?

Completed

Plan saved and checked

Needs the user

Missing dates? Ask, then pause.

Budget reached

12 rounds used. Report unfinished work.

Source: NanoBot @ 2245fc0 · teaching implementation

Asking the user is also a tool

```
if call.name == "ask_user":                    # a preference needs the user
    state.pending_call = call                  # keep the question and its ID
    sessions.save(state)                       # save the current progress
    return WaitingForUser(call.question)       # show it, then pause
```

INPUT

ask_user(question=
"What would you like to eat?")

OUTPUT

Question: “What would you like to eat?”
Status: waiting for user

Source: Teaching extension · ask_user pauses the task

Two budgets control different things

Trip budget

CNY 6,000
Limits the proposed plan.

Execution budget

12 agent rounds
Limits time and API work.

Hitting the execution limit means incomplete.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

The adapter translates requests and replies

What changes?	OpenAI Responses	Anthropic Messages
System / history	instructions / input	system / messages
Tool schema	parameters	input_schema
Tool call	function_call	tool_use
Call arguments	arguments: JSON string	input: object
Tool result	function_call_output	tool_result in user message
Match result to call	call_id	tool_use_id

NanoBot returns LLMResponse + ToolCallRequest to the same agent loop.

Source: NanoBot provider implementations / official tool-use APIs

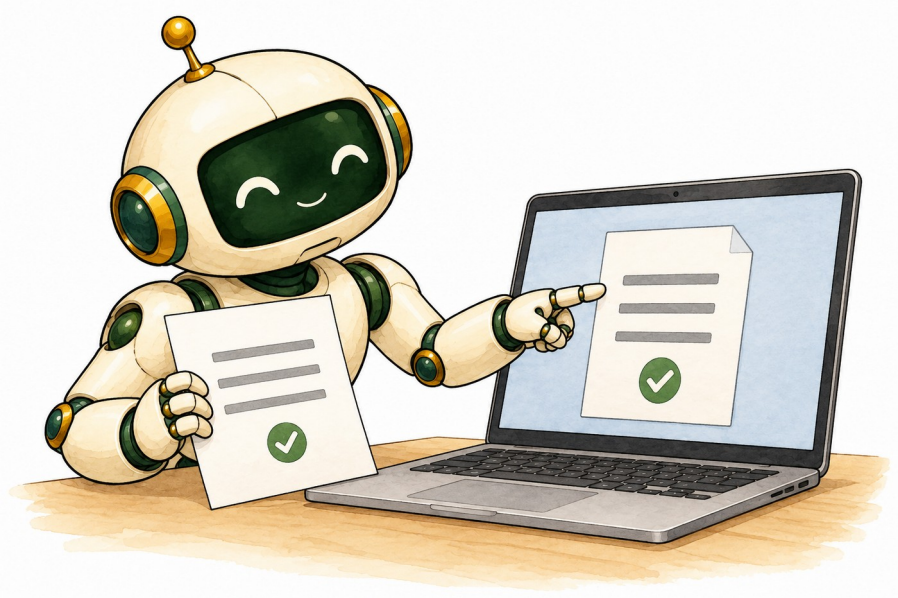
香 港 大 學
THE UNIVERSITY OF HONG KONG

PERSONAL AGENTS · LECTURE 6

03

Tools

PART 3 OF 6



Tools for this task

web_search	Compare flights, trains and hotels
web_fetch	Check fees and full page details
read_file	Read requirements or check the saved plan
write_file	Save plans/hong-kong.md

Source: NanoBot @ 2245fc0 · web.py

A file path is only the starting point



1 User message

“Plan my trip using project/trip.md.”

2 Agent tool call

```
read_file("project/trip.md")
```

3 Tool result enters the context

20–23 Nov • Shanghai to Hong Kong
1 person • Budget: CNY 6,000
Compare flights, trains and hotels.

Source: NanoBot @ 2245fc0 · teaching implementation

The model sees the tool definition

```
{
  "name": "write_file",
  "description": "Save text to a file",
  "parameters": {
    "type": "object",
    "properties": {
      "path": {"type": "string"},
      "content": {"type": "string"}
    },
    "required": ["path", "content"]
  }
}
```

Name
The model requests this name

Parameters

path + content
Both are required

Example path

plans/hong-kong.md

Source: NanoBot @ 2245fc0 · base.py

The developer registers the available tools

```
registry = ToolRegistry()                          # developer setup
registry.add("read_file", read_file)               # read local requirements
registry.add("web_search", web_search)             # find travel options
registry.add("web_fetch", web_fetch)               # read offer details
registry.add("write_file", write_file)             # save the final plan
tool = registry.get(call.name)                     # pick the requested tool
```

INPUT

call.name = "web_fetch"

OUTPUT

The web_fetch implementation

Source: NanoBot @ 2245fc0 · registry.py

Lecture 5 → our tools.py

```
Tool call from the agent
{name: "web_search", arguments: {...}}
```

tools.py routes the call

Python function

```
search(query)
```

CLI command

```
run(["search-cli",
query])
```

MCP server

```
client.call_tool(name,
args)
```

CLI: Command-Line Interface · MCP: Model Context Protocol

Source: Lecture 5 callback · function calling / MCP / CLI

The application runs the tool code

```
async def write_file(path, content):                    # developer-written function
    target = resolve_allowed_path(path)                 # stay inside allowed folders
    write_text(target, content)                         # write the Markdown bytes
    return {"saved": str(target)}                       # report success after writing
```

INPUT

path: plans/hong-kong.md
content: four-day plan + budget + sources

OUTPUT

{"saved": "plans/hong-kong.md"}

Source: NanoBot @ 2245fc0 · filesystem.py

Search returns candidates

```
web_search("Hong Kong Hotel A 20–23 Nov 3 nights")
```

Site A

Hotel A, 3 nights: CNY 2,700, fees included
https://example.com/hotel-a/site-a

Site B

Hotel A, 3 nights: from CNY 2,400
https://example.com/hotel-a/site-b

Source: Classroom scenario · all prices, times and tool outputs are illustrative

Fetch reads the details behind a result

```
web_fetch("https://example.com/hotel-a/site-b")
```

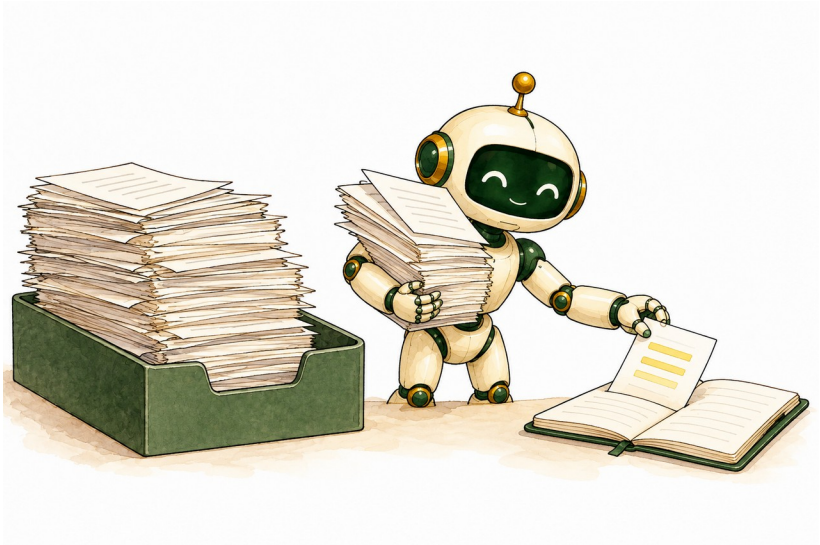
Hotel A / 3 nights

Room price: CNY 2,400
Taxes and fees: CNY 450
Total due: CNY 2,850

The headline price missed CNY 450.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

A web page can be too large for one call



Keep

Price, dates, fees and source

Reduce

Menus and repeated text

Source: NanoBot @ 2245fc0 · web.py

A typo can lead to the wrong search

Requested query

Hong Gong hotels

Result: no useful matches

Corrected query

Hong Kong hotels

The city now matches the request.

The next round uses the failed result to correct the input.

Source: Classroom scenario · all prices, times and tool outputs are illustrative

Tool errors return to the agent

```
try:                                                    # attempt the requested search
    result = await registry.run(call)                   # use the supplied query
except ToolError as error:                              # capture a real tool failure
    result = {"error": str(error)}                      # keep a useful reason
state.add(tool_result(call.id, result))                 # return feedback with its ID
```

INPUT

call_hotels: web_search("Hong Gong hotels")

OUTPUT

call_hotels: {"matches": []}
Or {"error": "Search timed out"}

Source: NanoBot @ 2245fc0 · registry.py

The saved plan is the task output

```
# Hong Kong: 4 days
Day 1: arrive, Tsim Sha Tsui waterfront
Day 2: Central and the Peak
Day 3: Lantau
Day 4: a short morning walk, then return

Budget: CNY 5,200 + CNY 800 reserve
Sources: transport and hotel links
```

Read it back

4 days?
Within CNY 6,000?
All fees counted?
Source links saved?

Source: Classroom scenario · all prices, times and tool outputs are illustrative

A Skill saves the planning method



Explain the method once

Compare total costs.
Check dates and fees.
Save the evidence.

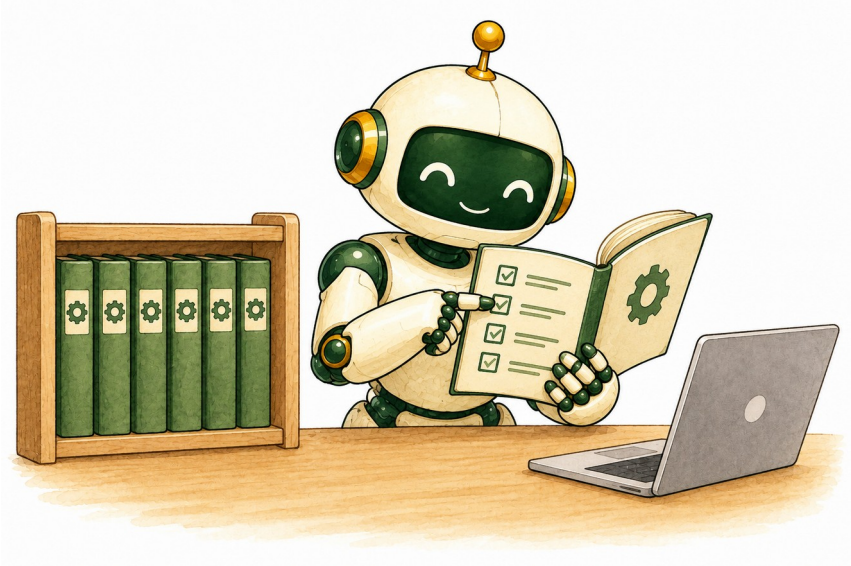
Source: NanoBot @ 2245fc0 · skills.py

A travel-planning Skill

```
---
name: travel-plan
description: Compare options within a travel budget
---
1. Confirm dates, people and budget.
2. Compare full transport and hotel costs.
3. Read the fees and cancellation terms.
4. Save the route, budget and source links.
5. Read the plan back and check the constraints.
```

Source: NanoBot @ 2245fc0 · skills.py

The agent loads the Skill when needed



First

Names and short descriptions

For this task

Read travel-plan/SKILL.md

Source: NanoBot @ 2245fc0 · skills.py

Skill discovery and loading

```
skills = discover_skill_files()                  # find available guides

context.add(catalog(skills))                     # add names + descriptions

selected = choose_skill(task, skills)            # choose travel-plan

if selected:                                     # only load a relevant guide

    context.add(read_skill(selected))            # include its full method
```

INPUT

Task: compare Hong Kong travel options
Skill catalog on disk

OUTPUT

Context includes travel-plan instructions

Source: NanoBot @ 2245fc0 · skills.py

MCP tools can use the same registry

```
definitions = await mcp.list_tools()                    # discover a server's tools

for definition in definitions:                          # wrap each available tool

    registry.add(MCPTool(definition, mcp))              # reuse our internal interface

result = await registry.run(call)                       # wrapper sends tools/call
```

INPUT

MCP: Model Context Protocol
A server exposes tools

OUTPUT

The same agent loop can request them

Source: NanoBot @ 2245fc0 · mcp.py

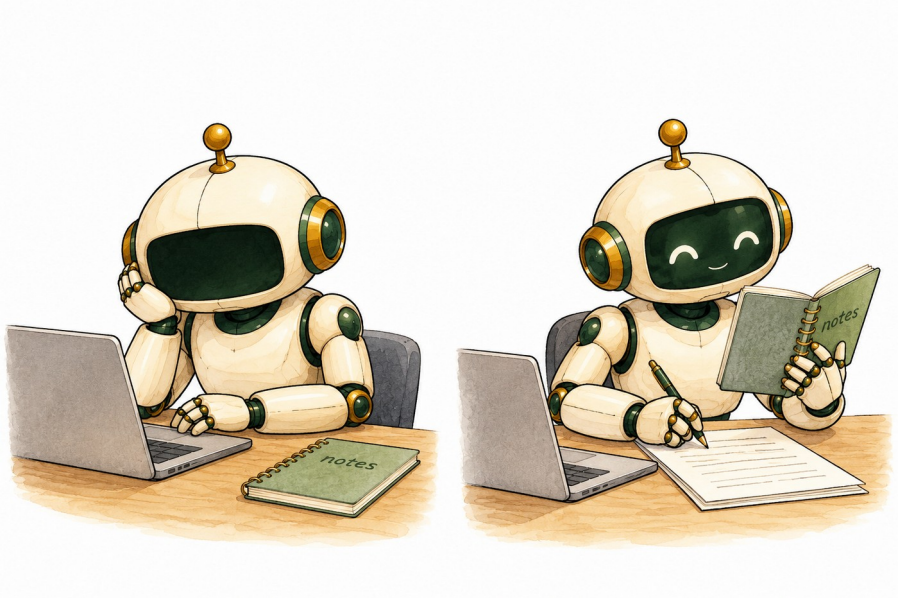
香 港 大 學
THE UNIVERSITY OF HONG KONG

PERSONAL AGENTS · LECTURE 6

04

Memory Management

PART 4 OF 6



Lecture 5: three useful memory timescales



Source: Lecture 5 callback · teaching categories, not a vendor standard

Codex: background work prepares memory

Optional feature: off by default

How local Codex memories work

After you enable memories, Codex can turn useful context from eligible prior chats into local memory files. Codex skips active or short-lived sessions, redacts secrets from generated memory fields, and updates memories in the background instead of immediately at the end of every chat.

Past chats	Background work	Future context
Wait until idle	**Extract + consolidate**	**Load saved memory**

The app prepares local memory in the background.

Source: OpenAI · local memories (opt-in)

Claude Code: a small index, then topic files

```
MEMORY.md

Travel → travel.md
Project → project.md
```



```
travel.md

- Vegetarian food
- Quiet hotel room
- Ask before booking
```

Startup: first 200 lines / 25 KB

Read the topic when needed

The model sees the index first, not every memory file.

Source: Claude Code · official memory documentation

The task changes. The useful memory changes.

CHOOSE A HOTEL

CNY 6,000 budget
Hotel A was too noisy

Need: budget + past rejection

FIND DINNER

Vegetarian food
Near today’s route

Need: preference + location

Source: Teaching design question · task-aware retrieval

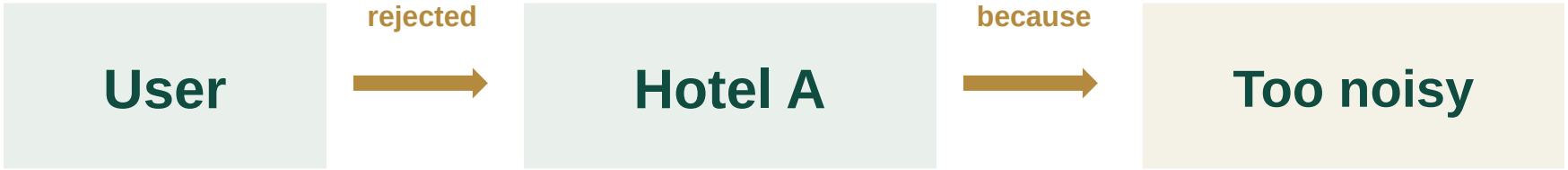
Keywords, embeddings, or a graph?

Method	A useful query	Main risk
Keywords	“Hotel A”	Different wording
Embeddings	“A peaceful place to stay”	Similar, but wrong
Graph + time	“Why did I reject that hotel?”	Wrong or stale links

Choose for the task. Read the evidence before answering.

Source: Graphiti / Hermes / DeepSeek · retrieval design choices

A graph can keep the relation



Evidence: “Hotel A was too noisy.” · Date: last trip

Source: Graphiti · temporal knowledge graphs / teaching adaptation

Memory should be revised

“This time, a quiet room matters more than the price.”

OLD NOTE

Always choose the cheapest.

→

REVISED, WITH SCOPE

For this trip: quiet first.

Keep this trip’s preference separate from a lifelong rule.

Source: MemEvolve · self-evolving agent memory / teaching example

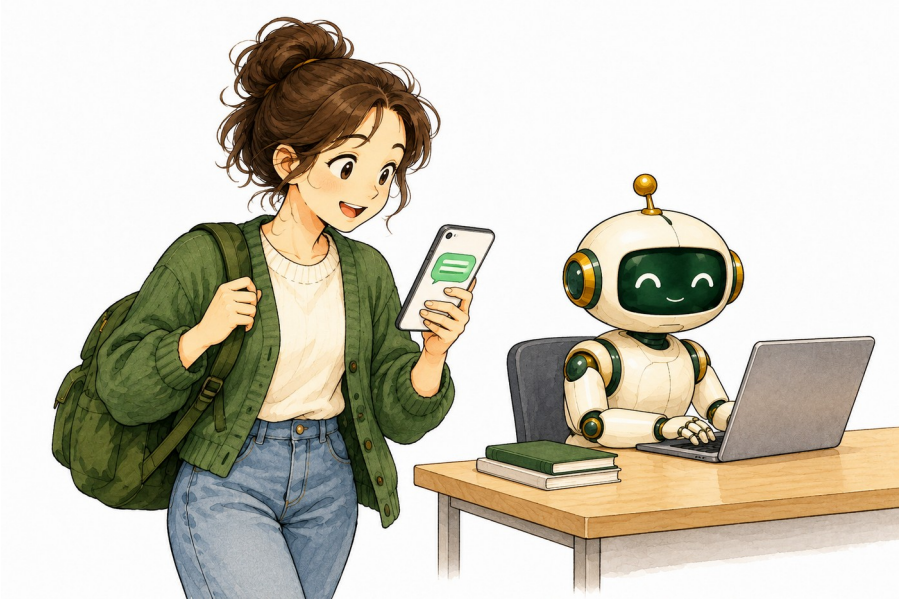
香 港 大 學
THE UNIVERSITY OF HONG KONG

PERSONAL AGENTS · LECTURE 6

05

Summary

PART 5 OF 6



The whole personal-agent system

User request + saved session

Read / save



Tool call + result return to the next context

Runtime keeps these parts connected.

Source: NanoBot @ 2245fc0 · teaching implementation